

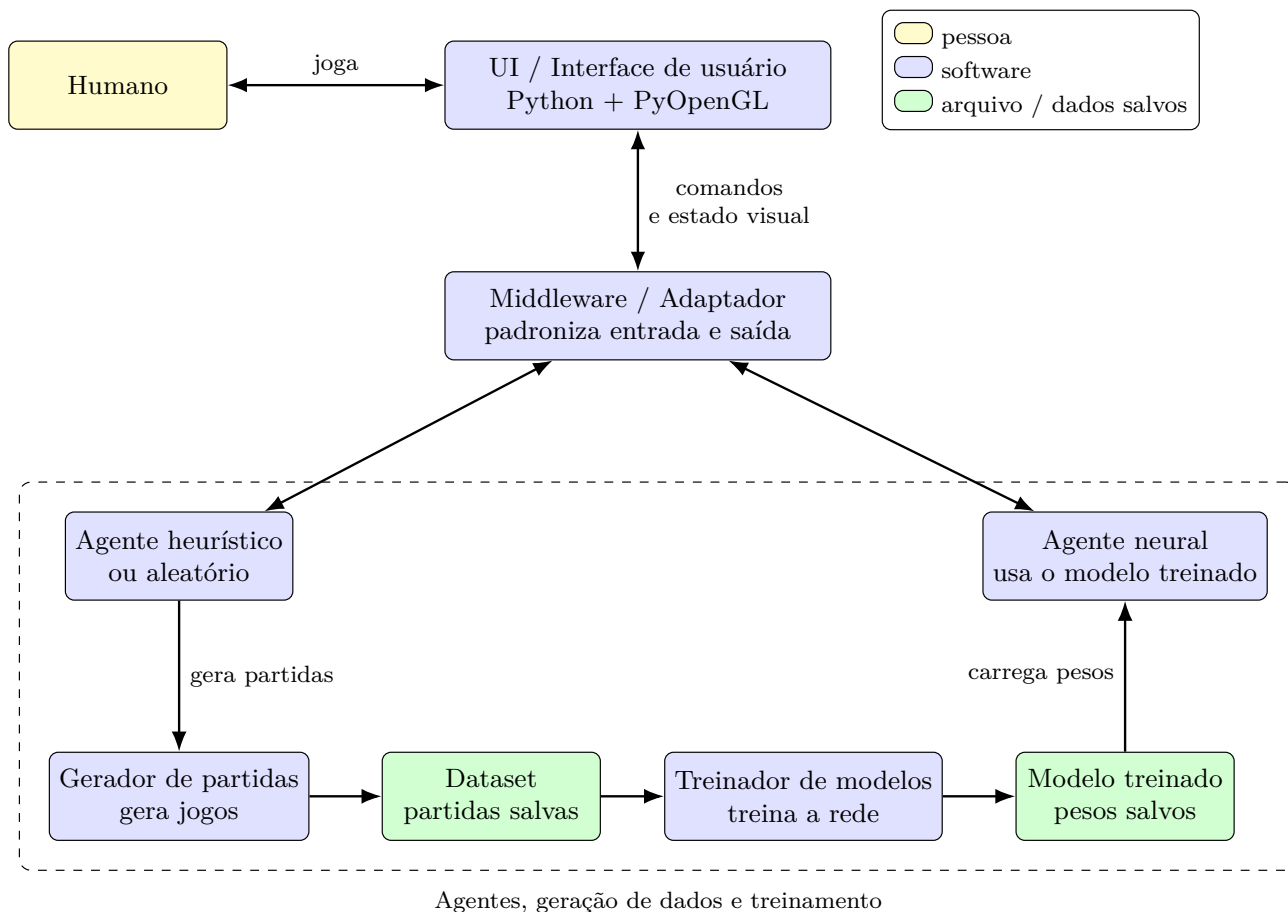
Arquitetura geral para uma rede neural aprender a jogar dominó

Ideia geral

A proposta é separar o projeto em módulos simples e independentes. A interface de usuário cuida apenas da parte visual. O humano interage diretamente com a UI. Os agentes computacionais não falam com a tela diretamente: eles passam por uma camada intermediária, aqui chamada de *middleware* ou *adaptador*. Essa camada padroniza entrada e saída, para que a UI consiga conversar do mesmo jeito com um jogador humano, um agente heurístico ou um agente neural.

Além da parte de jogo, é preciso separar a parte de treinamento. O modelo treinado é uma coisa; o programa que treina esse modelo é outra. Também convém ter um gerador de dataset de partidas, capaz de produzir muitas partidas automaticamente usando um agente heurístico.

Diagrama geral da arquitetura



Módulos principais

1. **Humano.** Fica fora do bloco dos agentes. O humano joga diretamente pela interface gráfica.
2. **UI / interface de usuário.** Deve permitir partidas locais no mesmo computador com 2, 3 ou 4 jogadores. Cada posição pode ser ocupada por um humano ou por uma IA. A interface mostra a mesa, as pedras, de quem é a vez, as jogadas e o resultado.
3. **Middleware / adaptador.** É a camada que faz a ponte entre a UI e os agentes. Ela transforma eventos da interface em um formato que os agentes entendem e transforma as decisões dos agentes em algo que a interface consegue mostrar. A ideia é uniformizar a comunicação.
4. **Agente heurístico.** É o jogador automático mais simples. Pode escolher uma jogada aleatória ou a primeira jogada legal encontrada. Serve como linha de base e também como ferramenta para gerar partidas.
5. **Gerador de partidas.** Usa o agente heurístico para produzir muitas partidas automáticas. O papel dele é jogar em grande quantidade e registrar o que aconteceu em cada partida.
6. **Dataset de partidas.** É o arquivo, ou conjunto de arquivos, produzido pelo gerador de partidas. Deve guardar estados do jogo, jogadas realizadas, jogador da vez, resultados finais e outras informações úteis para treinamento.
7. **Treinador de modelos.** É o programa que pega o dataset de partidas, monta exemplos de treino, atualiza os pesos da rede neural e salva versões do modelo treinado.
8. **Modelo treinado.** É o artefato final salvo em arquivo, contendo os pesos da rede. Ele não treina nada por si só; apenas é carregado depois para ser usado em jogo.
9. **Agente neural.** Usa o modelo treinado para escolher jogadas durante a partida. Em outras palavras, o agente neural é o “jogador” e o modelo treinado é o conhecimento que esse jogador consulta.

Fluxo recomendado de desenvolvimento

- Etapa 1.** Fazer uma primeira UI simples, só para permitir partidas locais no mesmo computador.
- Etapa 2.** Implementar o middleware, definindo um protocolo único de comunicação entre interface e agentes.
- Etapa 3.** Implementar o agente heurístico e garantir que ele consiga jogar partidas completas sem travar.
- Etapa 4.** Implementar o gerador de dataset, usando o agente heurístico para criar centenas, milhares ou milhões de partidas.
- Etapa 5.** Implementar o treinador da rede neural, consumindo o dataset e salvando modelos treinados.
- Etapa 6.** Implementar o agente neural, que carrega um modelo treinado e passa a jogar usando esse modelo.
- Etapa 7.** Comparar o agente neural com o agente heurístico e ir refinando o processo de treinamento.

Princípio de organização

Todos os agentes computacionais devem obedecer à mesma interface conceitual, por exemplo:

$$\text{escolher_jogada}(\text{estado}, \text{jogadas_legais}) \longrightarrow \text{jogada}.$$

Assim, a UI e o middleware não precisam saber se estão lidando com um agente heurístico ou com um agente neural. Basta chamar o mesmo protocolo. Isso simplifica bastante a organização do projeto.